Engineering best practices

Knowledge base 01 · Standards and guardrails we follow

Module	Subscription Management
Owner	Engineering team
Version	1.0, for review
Date	8 October 2026

Document control

Document	Subscription Management - KB 01 - Best Practices
Subject	Knowledge base 01 · Standards and guardrails we follow
Version	1.0
Date	2026-10-08
Status	For review
Owner	Engineering team
Generated by	`src/generators/emit-client-pack.js`
Source	`docs/client-pack/07-best-practices.md`
Source blueprint	`49f2c22e6c8f`
Prepared by	Ascendion
Classification	Ascendion Confidential

Contents

1. Purpose and scope
2. Principles
3. Secure development
4. Database engineering
5. Testing
6. AI-assisted engineering
7. Source control and delivery
8. Documentation
9. References

1. Purpose and scope

The engineering practices every engineer and agent follows on a Salesforce exit, and the industry references behind them. We check them automatically on every change. How we apply them to this module is in the HLD and LLD.

2. Principles

Principle	Practice
Single responsibility	One agent, one job, one output folder
One source of truth	A versioned model drives everything we generate; nobody edits generated output by hand
Deterministic and repeatable	The same input always gives the same output; CI regenerates and fails on any difference
Least privilege	Read-only access to the source; the narrowest role for every user, service and pipeline
Evidence over assertion	Every claim (a count, a match, a pass) comes from a report that anyone can re-run

3. Secure development

Risk area	Practice
Broken access control	Enforce access where the data lives, not only in screens; test each role
Injection	Parameterised queries only; no SQL built from user input
Secrets	Never in source; held in a vault, referenced by managed identity, scanned on every change
Software supply chain	Pinned dependency versions, lock files committed, pipeline actions pinned, audit on every build
Integrity	Protected main branch; nothing merges without review and passing checks
Logging and monitoring	Every agent run, approval and deployment is recorded with who and when

4. Database engineering

- Schema changes are numbered scripts, applied in order, each with one owner, rebuildable from the model
- Business rules as constraints, so every client of the data gets the same rule
- Names come from the business vocabulary, in one consistent case style
- Every change is backward compatible during a migration window (expand, then contract)

5. Testing

- The source system is the reference: expected values come from it (golden-master testing)
- Every behaviour has a test; nothing is released with a failing test
- Tests are code, versioned with what they test, and run in CI on every change
- Security has its own tests: each role tries what it may and may not do

6. AI-assisted engineering

Guardrail	How we enforce it
AI output is a proposal	An engineer reviews and approves it before it merges
The model is never in the runtime path	AI writes code at build time; the running system has no AI dependency
Each agent stays in its lane	Each agent may only write to its own folder
Human approval at every phase	Approve, edit or return at each gate, fully audited
No secrets or personal data in prompts	Inputs are masked and scanned automatically

7. Source control and delivery

- Trunk-based development with short-lived branches and pull-request review
- Infrastructure as code for every environment; no manual changes in the portal
- The pipeline deploys with a federated identity, not a stored password
- Every release is traceable to a commit, a review and a test report

8. Documentation

- Architecture documents follow arc42, with C4 diagrams for context and containers
- Every significant decision has an architecture decision record (ADR)
- Documents are generated from the source, so they never go out of date

9. References

Reference	Publisher	Link
OWASP Top 10	OWASP Foundation	https://owasp.org/Top10/
Application Security Verification Standard (ASVS)	OWASP Foundation	https://owasp.org/www-project-application-security-verification-standard/

Reference	Publisher	Link
Secure Software Development Framework, SP 800-218	NIST	https://csrc.nist.gov/pubs/sp/800/218/final
Security hardening for GitHub Actions	GitHub	https://docs.github.com/en/actions/security-guides/security-hardening-for-github-actions
The Twelve-Factor App	Heroku / community	https://12factor.net/
arc42 architecture template	arc42	https://arc42.org/
The C4 model for visualising software architecture	Simon Brown	https://c4model.com/
Architecture decision records	ADR GitHub organisation	https://adr.github.io/
PostgreSQL: row security policies	PostgreSQL Global Development Group	https://www.postgresql.org/docs/current/ddl-rowsecurity.html
PostgreSQL: constraints	PostgreSQL Global Development Group	https://www.postgresql.org/docs/current/ddl-constraints.html
Triggers and order of execution	Salesforce	https://developer.salesforce.com/docs/atlas.en-us.apexcode.meta/apexcode/apex_triggers_order_of_execution.htm